Handling Imbalanced Data in Binary Classification: A Review and Comparative Experimental Study

Chi-Wei, PENG
114423004
114423004@cc.ncu.edu.tw

Ting-Chun, Lin
114423009
Tinalin@g.ncu.edu.tw

Chiu-Hsiang, Lin
114423049
114423049@cc.ncu.edu.tw

Chia-Chen, Lee
114423050
cclee@g.ncu.edu.tw

Abstract

Class imbalance remains a pervasive challenge in binary classification, yet most comparative studies confound the effects of the imbalance ratio (IR) with variations in feature dimension and dataset size. This study addresses this gap by adopting a strictly controlled experimental design using four variants of the Yeast dataset. Because these variants share an identical feature space, the design effectively isolates the impact of class distribution, revealing the intricate interplay between IR and intrinsic data complexity (e.g., class overlap and small disjuncts). To ensure a robust performance comparison, three families of techniques—data-level, algorithm-level, and hybrid methods—are systematically evaluated across three base classifiers. The empirical results yield crucial insights: at the data level, SMOTE+ENN achieves the highest G-Mean under severe imbalance; at the algorithm level, Cost-Sensitive XGBoost offers the optimal precision-recall balance, while AdaBoost suffers from an extreme trade-off; and among hybrid approaches, SMOTEBoost demonstrates the most robust stability under extreme IR. Furthermore, the experiments reveal that intrinsic data complexity often dictates performance degradation more decisively than the sheer IR. Across all levels, accuracy proves to be a deceptively misleading metric, reinforcing the necessity of imbalance-aware evaluation. These findings provide actionable, data-driven guidelines for selecting appropriate strategies based on the specific nature of the imbalance and real-world error tolerance.

Keywords

Class Imbalance; Binary Classification; Resampling Techniques; Cost-Sensitive Learning; Ensemble Learning; Data Complexity.

1 Introduction

1.1 Background and Problem Definition

In recent years, the rapid expansion of data availability across large-scale, complex, and networked systems has propelled machine learning to prominent roles in numerous application domains [1]. However, real-world data distributions are inherently complex and frequently exhibit a significant class imbalance. The class imbalance problem arises when the number of instances belonging to one class (the majority class) substantially outnumbers those of another (the minority class), resulting in a severely skewed training distribution [1, 2]. This phenomenon is pervasive in many high-stakes domains, including financial fraud detection, network intrusion detection, and the diagnosis of rare diseases such as diabetes and stroke, where the minority class—despite its low frequency of occurrence—typically constitutes the primary target of predictive interest [3, 4].

1.2 Challenges and the Pitfalls of Evaluation Metrics

Standard machine learning models are generally designed under the implicit assumption of an approximately balanced class distribution. Consequently, when applied to skewed data, these models exhibit a systematic bias toward the majority class, yielding poor predictive performance for minority instances—precisely those of greatest practical concern [1]. This issue is further compounded by the widespread use of conventional evaluation metrics. Overall accuracy, in particular, is highly misleading under class imbalance: a naive classifier that assigns all instances to the majority class can attain superficially high accuracy while entirely failing to identify any minority-class example [5]. To overcome this limitation, the research community has advocated the adoption of more discriminative metrics, including the F1-Score, the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) [6], and the Geometric Mean (G-Mean) [7], which collectively provide a more faithful assessment of model performance across both classes.

1.3 Taxonomy of Imbalanced Learning Techniques

To address the challenges posed by imbalanced datasets, a rich body of literature has proposed numerous techniques, which are broadly categorized into three main families: data-level methods, algorithm-level methods, and hybrid strategies [1, 8]. Data-level approaches modify the training data distribution prior to model induction, either by oversampling the minority class—most notably through the Synthetic Minority Over-sampling Technique (SMOTE) [2] and its variants—or by under sampling the majority class through methods such as Random Under-Sampling(RUS). Algorithm-level approaches, by contrast, leave the data distribution unchanged and instead modify the learning procedure itself; representative techniques include cost-sensitive learning and ensemble-based methods such as weighted classifiers and boosting frameworks [9]. Finally, hybrid strategies combine the complementary strengths of both data resampling and algorithm robustness, with methods such as SMOTEBoost and RUSBoost serving as prominent examples [9].

1.4 Research Gap and Objectives

Despite the substantial volume of existing work, many comparative studies evaluate imbalanced learning techniques across heterogeneous datasets that vary simultaneously in multiple dimensions—including class overlap, feature complexity, and sample size—making it difficult to isolate the independent effect of the Imbalance Ratio (IR) on classifier performance [1]. Confounding factors of this nature limit the interpretability of empirical findings and reduce

their practical utility as guidelines for practitioners. To address this gap, this study proposes a rigorous empirical framework grounded in controlled experimental design. Rather than relying solely on a diverse collection of independent datasets, we systematically vary the IR within a curated set of benchmark datasets spanning a wide range of imbalance severity—from mild (IR ≈ 2.4) to extreme (IR ≈ 41.4)—thereby enabling direct, controlled comparisons across escalating levels of class imbalance. Within this framework, we conduct a systematic evaluation of representative methods from all three technique families—data-level, algorithm-level, and hybrid—using three established base classifiers (SVM, Random Forest, and XGBoost) and a comprehensive set of evaluation metrics (F1-Score, AUC-ROC, and G-Mean). By analyzing the performance degradation trajectories of each strategy as class imbalance worsens, this study aims to provide both a nuanced empirical understanding of the strengths and limitations of existing approaches and actionable practical guidance for selecting appropriate imbalanced learning techniques in binary classification tasks.

2 Method

2.1 Data-level Methods

At the data pre-processing stage, techniques for addressing class imbalance can be broadly categorized into two types: non-synthetic and synthetic methods. Non-synthetic methods adjust the class distribution using existing data, leaving the underlying information unaltered. Conversely, synthetic methods mitigate the imbalance by generating new data samples using existing features.

2.1.1 Non-synthetic method.

(i) Random sampling: Resampling strategies generally include techniques such as under-sampling, over-sampling, and hybrid sampling. Random under-sampling(RUS) deletes samples from the majority class, which may discard useful data. Random over-sampling(ROS) achieves dataset balance by duplicating minority class samples, but this exact duplication increases the risk of overfitting.[10]

In most cases, under-sampling improves model performance more effectively than over-sampling. However, in certain specific scenarios—such as when the negative class is dominant—the opposite holds true. The optimal resampling ratio varies depending on the dataset and the chosen resampling strategy, rather than simply supplementing or reducing classes to a perfectly balanced state. [11]

(ii) Hybrid sampling: Hybrid sampling combines the advantages of both over-sampling and under-sampling while mitigating their respective disadvantages. Applying over-sampling before under-sampling generally yields better performance than the reverse order. However, the hybrid sampling ratio must be determined manually, often requiring domain knowledge. If this ratio is inappropriately set for a given dataset, the approach may fail to show the benefits of hybridization and could even perform worse than using either technique alone. [12]

2.1.2 Synthetic method.

(i) SMOTE. To overcome the limitations inherent in random resampling, the Synthetic Minority Over-sampling Technique (SMOTE) was introduced [13]. Instead of merely duplicating existing instances, SMOTE over-samples the minority class by creating synthetic examples. This mechanism effectively forces the decision regions of the minority class to be more extensive and generalized.

The specific mechanism of SMOTE involves taking each minority class instance and generating new synthetic samples by randomly interpolating points along the line segments connecting it to its K nearest minority-class neighbors. A significant drawback of this approach is its failure to consider the distribution of the majority class during synthesis. Consequently, this oversight can generate ambiguous samples and blur decision boundaries, particularly in scenarios with substantial inter-class overlap. [14]

Several SMOTE extensions have been proposed to address the shortcomings. Borderline-SMOTE solves the problem of blindly over-sampling all minority samples [15]. Furthermore, SMOTE+Tomek and SMOTE+ENN overcome the class overlapping and overfitting issues frequently caused by SMOTE [10]. In other words, Borderline-SMOTE focuses on selective generation, while SMOTE+Tomek and SMOTE+ENN focus on comprehensive data cleaning after synthetic examples are created.

(ii) Borderline-SMOTE. The Borderline-SMOTE algorithm operates in two primary stages. The first stage focuses on identifying boundary samples. Each minority class instance, p_i, is categorized into one of three types: Noise (excluded from synthesis), Safe (excluded from over-sampling), and Danger (targeted for over-sampling and added to the DANGER set).

The second stage involves generating synthetic samples. Boundary samples within the DANGER set are interpolated using a random number multiplier to create new instances. This stage employs two distinct strategies: Borderline-SMOTE1 exclusively utilizes nearest neighbors from the minority class, yielding highly balanced and superior performance in terms of True Positive (TP) rate and F-value. Conversely, Borderline-SMOTE2 incorporates neighbors from both the minority and majority classes. While this approach maximizes the detection rate of the minority class, it inherently risks creating overlapping feature spaces between the two classes.

(iii) SMOTE+Tomek and SMOTE+ENN.

(1) SMOTE+Tomek

A Tomek link is defined as a pair of instances from opposite classes that are mutual nearest neighbors [16]. The presence of such a link suggests that either one instance is noise, or both reside exactly on the decision boundary. The SMOTE+Tomek approach initiates by over-sampling the dataset via SMOTE, followed by the identification of Tomek links. In the subsequent data cleaning phase, both instances constituting a Tomek link are simultaneously removed to establish distinct and well-defined class boundaries.

(2) SMOTE+ENN

Similar to the SMOTE+Tomek links approach, SMOTE+ENN employs the Edited Nearest Neighbor (ENN) algorithm during the data cleaning phase. ENN is a noise removal technique based on the k-nearest neighbors rule [17]. Specifically, SMOTE+ENN examines each instance in the training set; if an instance is misclassified by the majority vote of its three nearest neighbors (k=3), it is identified as noise and subsequently eliminated from the dataset.

SMOTE+Tomek links involves finding pairs of samples from different classes that are mutual nearest neighbors. In contrast, SMOTE+ENN comprehensively considers the distribution of a sample's three nearest neighbors, eliminating the sample if the majority belong to the opposing class. Consequently, models trained with SMOTE+ENN usually show the smallest increase in average decision rules, indicating that it produces more streamlined models.

2.2 Algorithm-level Methods

Algorithm-level strategies aim to address class imbalance by directly modifying the internal learning mechanisms of machine learning algorithms, rather than altering the original dataset, so that the model places greater emphasis on minority class samples during training [18].

2.2.1 Cost-Sensitive learning.

Traditional classification algorithms typically assume that all misclassification errors carry equal cost, and therefore optimize toward maximizing overall accuracy. In imbalanced data settings, however, the cost of misclassifying a minority class sample is often substantially higher than that of misclassifying a majority class sample [19].

The core idea of Cost-Sensitive Learning is to introduce a Cost Matrix that assigns different penalty weights to misclassifications across different classes [19]. Through this mechanism, the model can be compelled to shift greater attention toward the minority class during training without modifying the original class distribution. This concept underpins the implementation approach of the three models discussed in the following sections.

2.2.2 Ensemble method.

The core idea of ensemble methods is to combine multiple weak learners to construct a more powerful model. This review focuses on two dominant frameworks: the Bagging framework, represented by Random Forest, and the Boosting framework, represented by XGBoost and AdaBoost [18].

(i) Random Forest (Bagging Framework). Random Forest improves model stability by constructing multiple decision trees in parallel and aggregating their predictions through majority voting [20]. In imbalanced data settings, however, each decision tree grows by minimizing overall Gini Impurity, which causes the model to prioritize the purification of majority class nodes. As a result, the decision boundaries for minority class samples tend to be overlooked [18].

(ii) XGBoost (Boosting Framework). XGBoost was proposed by Chen & Guestrin in 2016, and operates by sequentially adding new functions to the model to progressively correct its errors [20]. Unlike the parallel construction of Random Forest, XGBoost's learning process is driven by the loss function — the model determines the growth direction of each tree by iteratively minimizing this loss. However, since standard Log Loss applies equal misclassification penalties across all samples, the overwhelming proportion of majority class samples in imbalanced datasets causes the gradient update to be consistently dominated by the majority class. Even when minority class errors occur repeatedly, their contribution to the overall loss remains marginal, making it difficult for the model to effectively correct these prediction errors over successive iterations [21].

(iii) Adaboost (Boosting Framework). AdaBoost was proposed by Freund & Schapire (1995), and its core mechanism involves dynamically adjusting sample weights at each iteration based on the classification results of the previous round. Misclassified samples receive higher weights, directing the next tree to focus more on correcting these difficult cases [22]. In severely imbalanced datasets, however, although the weights assigned to misclassified minority class samples increase over iterations, the sheer volume of majority class samples means that the cumulative influence of the minority class remains relatively limited within the overall weight distribution, and the iterative process may still be dominated by the majority class [19].

2.3 Hybrid Methods

In binary classification problems with imbalanced datasets, relying solely on data-level resampling or algorithm-level strategies is often insufficient, particularly under extreme imbalance or overlapping decision boundaries. To address these limitations, many researchers have attempted to integrate both approaches in innovative ways to overcome their respective drawbacks. In this study, hybrid methods are defined as approaches that integrate data-level sampling or data generation strategies with algorithm-level learning frameworks, thereby exploiting the complementarystrengths of both techniques. The following sections review the evolution of Boosting-based hybrid methods and other representative hybrid approaches.

2.3.1 Boosting-based Hybrid Methods. Within ensemble learning methods, AdaBoost [18] is one of the most representative algorithms. However, its original design was not optimized for handling class imbalance and therefore tends to produce biased predictions toward the majority class. To address this issue, numerous studies over the past two decades have proposed hybrid approaches that embed data resampling techniques into the iterative Boosting process, forming a clear evolutionary trajectory.

(i) SMOTEBoost.

Chawla et al. (2003) [23] first proposed the pioneering SMOTEBoost algorithm. This method integrates SMOTE into the AdaBoost.M2 framework. By creating new samples in the feature space, SMOTEBoost effectively enlarges the decision region of the minority class, thereby mitigating the learning bias caused by the equal treatment of misclassification errors in standard Boosting. SMOTEBoost has demonstrated strong recall performance in domains such as network intrusion detection, medical diagnosis, and web mining. However, under highly overlapping data distributions, blindly generated synthetic samples may introduce noise and blur the decision boundary.

(ii) RUSBoost.

To overcome the limitations of SMOTEBoost, Seiffert et al. (2010) proposed RUSBoost [24]. In this method, random under-sampling of the majority class is performed prior to each Boosting iteration to rebalance the dataset. By significantly reducing the training data

size, RUSBoost achieves much faster training speed compared to SMOTEBoost while simultaneously avoiding the risk of overfitting and noise introduced by artificial samples. This method is particularly suitable for large-scale datasets, such as those encountered in software quality modeling. Empirical results demonstrate that its classification performance, measured by metrics such as AUC and G-mean, is comparable to or even superior to SMOTEBoost, establishing it as a new benchmark for handling imbalanced data at the time.

(iii) EUSBoost and RHSBoost.

Subsequently, Galar et al. (2013) introduced EUSBoost [25], which aimed to address the randomness inherent in traditional sampling strategies by incorporating intelligent evolutionary mechanisms. In this method, evolutionary algorithms replace purely RUS, and a CHC-based genetic algorithm is employed to intelligently search for and retain the most representative majority-class samples during each iteration. Furthermore, the fitness function is modified to enforce diversity among classifiers. EUSBoost has achieved notable improvements in applications such as anomaly detection, facial age estimation, and image annotation, and has been shown to outperform RUSBoost on highly imbalanced datasets.

In 2017, Gong and Kim proposed RHSBoost [26], which integrates multiple sampling strategies within a single framework. In each AdaBoost iteration, majority samples are first reduced via RUS, followed by ROSE, which employs kernel density estimation to generate samples with variability without increasing the total dataset size. This bidirectional sampling strategy enables RHSBoost to produce clearer decision boundaries compared with single sampling approaches while avoiding significant increases in training cost. The method has shown advantages in applications such as financial risk management, corporate default prediction, and rare disease diagnosis.

(iv) WHMBoost.

Entering the 2020s, Zhao et al. (2020) proposed WHMBoost [27], shifting the focus from single sampling techniques toward weighted ensemble strategies. This framework constructs multiple balanced subsets at the data level using SMOTE and an improved adjustable random balance strategy. Decision trees and support vector machines are then alternately used as base classifiers. By evaluating the predictive capability of each classifier on positive and negative samples, adaptive weights are assigned dynamically. Through the integration of hybrid data processing and advanced algorithmic weighting mechanisms, WHMBoost demonstrates superior performance in credit card fraud detection and various benchmark datasets.

(v) SUBoost.

Continuing the exploration of information loss caused by under-sampling, Rasi Baghmishe et al. (2025) proposed the novel SUBoost method [28]. Unlike traditional approaches that rely on RUS, SUBoost adopts a selective under-sampling strategy. Specifically, it removes only those majority-class samples that are considered redundant or noisy, while retaining all minority samples and difficult majority samples that are frequently misclassified. Experimental results across more than 20 standard datasets show that SUBoost achieves superior performance in terms of AUC and G-mean compared with many mainstream hybrid methods, including RUSBoost and SMOTEBoost. The method has demonstrated strong robustness in applications such as cancer detection, financial fraud detection, and handwritten digit recognition.

2.3.2 Other Hybrid Approaches . Beyond Boosting-based ensemble learning, researchers have also developed hybrid strategies that combine data-level techniques with other algorithmic frameworks.

(i) S-CSL .

Researchers have recognized that relying solely on data resampling or internal classifier cost adjustments has inherent limitations. To address this issue, Thai-Nghe et al. proposed an empirical framework known as S-CSL [29]. In this approach, SMOTE is first applied during the data preprocessing stage to moderately expand minority samples. Subsequently, an asymmetric cost matrix is introduced at the algorithm level to significantly increase the penalty for misclassifying minority instances. S-CSL effectively mitigates the tendency of SVM hyperplanes to shift toward the minority class when dealing with imbalanced data. Experiments conducted on 18 imbalanced datasets, including disease diagnosis and academic performance prediction tasks, demonstrate that the method significantly reduces the overall misclassification cost.

(ii) CTGAN-MOS.

With the rapid development of deep learning techniques, Majeed and Hwang (2023) proposed CTGAN-MOS [30]. As a novel hybrid framework, CTGAN-MOS integrates six key stages, including data engineering, vulnerability analysis, high-quality synthetic data generation using a CTGAN model, and an intelligent fusion process. Unlike traditional distance-interpolation approaches such as SMOTE, CTGAN-MOS employs entropy optimization to generate only a small number of highly representative minority-class samples. Subsequently, a coin-flipping algorithm is used to eliminate noisy synthetic records, addressing the common issue in generative models where unsafe samples are produced within majority-class regions. CTGAN-MOS has been widely applied in safety-critical tasks such as adult income prediction, stroke prediction, credit fraud detection, and COVID-19 diagnosis.

3 EXPERIMENTS

3.1 Datasets

To rigorously investigate the effect of varying Imbalance Ratios (IR) on classifier performance in isolation from confounding factors, this study employs four benchmark datasets derived from the Yeast protein localization dataset, all sourced from the KEEL (Knowledge Extraction based on Evolutionary Learning) repository [31]. The Yeast dataset family is selected as the experimental foundation because all four variants share an identical feature space (8 features) and total sample size (n = 1,484), thereby ensuring that IR is the sole varying experimental variable. This controlled design eliminates confounds commonly present in cross-dataset comparisons, such as differences in feature complexity, dimensionality, and data size [8], enabling a direct and rigorous assessment of how escalating class imbalance affects classifier performance.

The Imbalance Ratio is defined formally as:

$$\mathrm{IR} = \frac{|S_{\mathrm{maj}}|}{|S_{\mathrm{min}}|}, \tag{1}$$

where $|S_{\mathrm{maj}}|$ and $|S_{\mathrm{min}}|$ denote the number of instances in the majority and minority classes, respectively. Following the categorization proposed in the imbalanced learning literature [1], the four datasets are stratified across four severity levels: *mild* (IR $<$ 3), *moderate* ($3 \leq$ IR $<$ 15), *severe* ($15 \leq$ IR $<$ 30), and *extreme* (IR $\geq$ 30), with each Yeast variant representing one distinct level.

All experiments employ **stratified 5-fold cross-validation** to ensure that the class distribution is preserved in every fold. The complete characteristics of the benchmark suite are summarised in Table[1].

Note: All four datasets originate from the same Yeast protein localization source and share an identical feature space, ensuring that IR is the sole experimental variable across conditions.

The four Yeast variants span IR values from 2.46 to 41.40, covering nearly a 17-fold increase in class imbalance severity. This progression allows the systematic observation of performance degradation trajectories across all three families of imbalanced learning techniques evaluated in this study — data-level, algorithm-level, and hybrid methods — under strictly controlled experimental conditions.

3.2 Experimental Setup

To ensure the reproducibility of the experimental results, all stochastic processes in the experimental pipeline are controlled using a fixed random_state value of 8.

3.2.1 Data Level.

In terms of the experimental design, four data-level approaches—the baseline (original data), ROS, SMOTE, and SMOTE+ENN—were evaluated across three distinct classifiers: K-Nearest Neighbors (KNN), Random Forest, and Support Vector Machine (SVM) [32]. Specifically, both ROS and SMOTE are configured to increase the minority class to achieve a 1:1 class ratio. In contrast, SMOTE+ENN first balances the classes to a 1:1 ratio and subsequently performs a data reduction phase to remove instances. Given its status as a cornerstone method in imbalanced learning, SMOTE serves as the central benchmark. The comparison among SMOTE, the baseline, and ROS validates SMOTE's efficacy in addressing fundamental class imbalance issues. Conversely, contrasting SMOTE with the SMOTE+ENN method effectively illustrates the inherent limitations of SMOTE across these different learning algorithms.

3.2.2 Algorithm Level.

Six experimental models were designed at the algorithm level, organized into three comparative pairs: standard Random Forest, standard XGBoost, and standard AdaBoost serve as baselines, each corresponding to a Cost-Sensitive counterpart — Weighted Random Forest, Cost-Sensitive XGBoost, and Cost-Sensitive AdaBoost, respectively. The following describes the Cost-Sensitive parameter settings and their corresponding mathematical mechanisms for each model.

(i) Weighted Random Forest. Weighted Random Forest addresses class imbalance by assigning different penalty weights to each class, directing the decision trees to place greater attention on minority class samples during the growing process [33]. In this study, the mechanism is implemented by setting class_weight to balanced, which allows the model to automatically compute class weights based on the inverse of each class's sample frequency.

(ii) Cost-Sensitive XGBoost. Cost-Sensitive XGBoost is implemented by setting scale_pos_weight to the Imbalance Ratio of the dataset, which proportionally amplifies the first-order gradient of minority class samples within the loss function [21]. This configuration directs the model to prioritize the correction of minority class errors at each iteration, achieving a cost-compensation effect within the Boosting framework [18].

(iii) Cost-Sensitive AdaBoost. Cost-Sensitive AdaBoost is implemented by setting the class_weight of the base weak learner (Decision Stump) to balanced, which causes each decision tree to assign higher impurity penalty weights to minority class nodes during growth. This ensures that the Boosting iterative process treats the correction of minority class errors as a priority from the very first round [22].

Beyond the Cost-Sensitive parameters described above, all remaining hyperparameters for each model were set to the default values provided by the scikit-learn and XGBoost libraries [21]. This ensures a consistent comparison baseline across models, so that any differences in experimental results can be attributed to the model architecture and Cost-Sensitive mechanism itself, rather than to differences in hyperparameter tuning strategies.

3.2.3 Hybrid Methods.

To comprehensively evaluate the effectiveness of data-level sampling techniques and algorithm-level cost mechanisms, this study designs two core experiments focusing on different hybrid learning strategies for imbalanced classification.

(i) Experimental Objectives. Two experiments are conducted to investigate different hybrid learning configurations. The evaluated methods include SMOTE, AdaBoost, SMOTEBoost, RUSBoost, RHSBoost, SUBoost, and SMOTECSL.

(1) Experiment 1: Data-level methods under a Boosting framework The first experiment examines the impact of various data-level sampling techniques when embedded within a Boosting framework. By integrating different sampling strategies into the Boosting iteration process, this experiment analyzes how data-level interventions influence the learning dynamics and minority-class detection performance. The evaluated approaches include Standard AdaBoost as the baseline method, as well as several hybrid variants, namely SMOTEBoost, RUSBoost, RHSBoost, and SUBoost.

(2) Experiment 2: Algorithm-level mechanisms combined with SMOTE When SMOTE is used as the primary data augmentation technique, the second experiment investigates which algorithm-level mechanism can further improve classification performance. In this experiment, SMOTE is used as a common preprocessing step, and the following configurations are compared: SMOTE combined with a baseline classifier, SMOTEBoost, and SMOTECSL.

Table 1: Summary of Benchmark Datasets.

Dataset	Instances	Features	Minority	Majority	Imbalance Ratio	Severity
Yeast-1	1,484	8	429	1,055	2.46	Mild
Yeast-3	1,484	8	163	1,321	8.10	Moderate
Yeast-4	1,484	8	51	1,433	28.10	Severe
Yeast-6	1,484	8	35	1,449	41.40	Extreme

(ii) Experimental Framework. To ensure fairness and reproducibility, this study does not directly rely on heterogeneous open-source implementations that may introduce hidden variations. Instead, a unified algorithmic testing framework was implemented based on scikit-learn and imblearn, with all algorithms reimplemented to align with the mathematical formulations described in the original literature [23, 24, 26].

All models were trained and evaluated within this unified framework to ensure that performance differences originate solely from the imbalance-handling mechanisms rather than implementation discrepancies.

(iii) Control Variables. To ensure that the experimental results are solely influenced by the imbalance-handling strategies, several variables are strictly controlled across all models.
(1) Base estimator alignment Traditional AdaBoost implementations commonly use decision stumps (max_depth = 1) [18]. However, shallow trees may fail to capture complex decision boundaries in imbalanced datasets. Following the experimental settings of SMOTEBoost, all Boosting-based models in this study adopt decision trees with max_depth = 5 as base classifiers [23–26].
(2) Unified AdaBoost engine Many machine learning libraries implement AdaBoost using the probabilistic SAMME.R variant. However, most imbalance-learning studies derive their theoretical formulations using discrete class labels. To maintain consistency with the original literature, all Boosting-based models in this study adopt the SAMME algorithm.

(iv) Literature-aligned Implementation Adjustments. Directly integrating existing Python implementations may cause deviations from the theoretical assumptions described in the original literature. Therefore, several implementation adjustments were introduced to preserve algorithmic consistency.
(1) Weight Isolation Mechanism If classification errors are calculated using resampled datasets, the theoretical convergence properties of AdaBoost may be compromised. To address this issue, a weight isolation mechanism is introduced.

Weak classifiers are trained on the resampled datasets, while classification errors, classifier weights (α_t), and sample weight updates are computed using predictions on the original training dataset. This design ensures that the Boosting weight update process remains theoretically valid.
(2) Decoupled Sampling Strategy in RHSBoost In many open-source implementations of RHSBoost, the default sampling configuration may reduce the majority class to a perfectly balanced ratio during the under-sampling stage. This behavior prevents the subsequent oversampling step from functioning effectively.

To preserve the intended hybrid design, the under-sampling and oversampling ratios are explicitly decoupled. Majority samples are first reduced while retaining sufficient information, and minority samples are subsequently expanded through oversampling.
(3) Cost Matrix Anchoring in SMOTECSL: When SMOTE is applied before cost-sensitive learning, the resulting balanced dataset may cause the classifier to assign equal class weights, thereby neutralizing the intended cost-sensitive mechanism [29].

To address this issue, the true cost matrix is computed from the original imbalanced labels prior to the SMOTE process and then explicitly mapped to the classifier loss function. This design ensures that feature-space expansion and algorithm-level cost penalties can operate simultaneously without canceling each other.

3.3 Evaluation Metrics

A central challenge in evaluating classifiers on imbalanced datasets is metric selection. As demonstrated by He and Garcia[1] and mathematically formalised by Luque et al.[5], overall *Accuracy* is a fundamentally flawed metric under class imbalance: a degenerate classifier that assigns every instance to the majority class can attain superficially high accuracy while entirely failing to detect any minority-class example. Accordingly, this study excludes Accuracy as a primary performance measure and instead adopts a suite of imbalance-aware metrics grounded in the binary confusion matrix, whose elements are defined as follows: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), where the *positive* class denotes the minority class throughout.

Precision and Recall. Precision measures the proportion of predicted positive instances that are truly positive; Recall (Sensitivity) measures the proportion of actual positive instances that are correctly identified. Both metrics are defined over the minority class and are expressed as[34]:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{3}$$

F1-Score. The F1-Score is the harmonic mean of Precision and Recall, providing a single scalar that penalises severe imbalances between the two[35]:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,TP}{2\,TP + FP + FN}. \tag{4}$$

Because the harmonic mean is strictly lower than the arithmetic mean whenever Precision and Recall diverge, F_1 constitutes a conservative and informative composite measure for imbalanced classification tasks.

AUC-ROC.. The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) is a threshold-independent metric that quantifies a classifier's ability to discriminate between classes across all

possible decision thresholds[6]. Let the False Positive Rate (FPR) and True Positive Rate (TPR) be defined as:

$$\text{FPR} = \frac{FP}{FP + TN}, \qquad \text{TPR} = \frac{TP}{TP + FN}. \tag{5}$$

The AUC-ROC is then computed as:

$$\text{AUC}_{\text{ROC}} = \int_0^1 \text{TPR}\big(\text{FPR}^{-1}(x)\big)\, dx. \tag{6}$$

A value of 0.5 corresponds to random classification, while 1.0 denotes a perfect classifier. As noted by Saito and Rehmsmeier[36], however, ROC-based metrics can present an overly optimistic picture under extreme class imbalance, because the large pool of true negatives in the majority class suppresses the FPR, thereby inflating the apparent curve.

G-Mean. The Geometric Mean (G-Mean), originally proposed by Kubat and Matwin[7] for imbalanced learning, is the square root of the product of Sensitivity (Recall) and Specificity. It provides a balanced assessment of performance across both classes simultaneously: poor performance on *either* class leads to a sharp decline in the overall score. Specificity and G-Mean are defined as:

$$\text{Specificity} = \frac{TN}{TN + FP}, \tag{7}$$

$$\text{G-Mean} = \sqrt{\text{Recall} \times \text{Specificity}} = \sqrt{\frac{TP}{TP + FN} \cdot \frac{TN}{TN + FP}}. \tag{8}$$

Metric Selection Rationale. Although this study reports all metrics described above for completeness, the **primary evaluation metrics are F1-Score, AUC-ROC, and G-Mean**. These three are selected because they collectively capture complementary dimensions of classifier performance under imbalance: the F1-Score reflects the precision–recall trade-off for the minority class; the AUC-ROC captures threshold-independent ranking ability across all decision boundaries; and the G-Mean ensures that majority-class performance is not sacrificed for minority-class gains. primary comparisons, in accordance with the limitations identified in prior literature[1, 5, 36].

4 Results

4.1 Data Level

This section presents the experimental results evaluated at the data level. The summary chart (Table 2) illustrates the average performance of the various data pre-processing methods across all classifiers, while the comprehensive experimental data can be found in the Appendix.

To facilitate an in-depth analysis utilizing the G-Mean as the core evaluation metric, the detailed results and visualizations are organized as follows:

- Tabular Results by Classifier: The detailed G-Mean performance for each classifier is presented in Table 3 (KNN), Table 5 (SVM), and Table 4 (Random Forest).
- Visualizations by Dataset: The comparative performance of the different data-level approaches across the three classifiers is graphically depicted by dataset in Figure 2 (yeast1), Figure 3 (yeast3), Figure 4 (yeast4), and Figure 5 (yeast6).

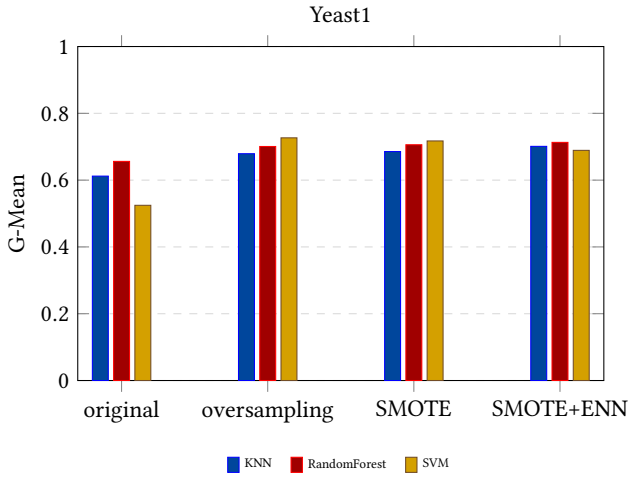

Figure 1: G-Mean between different techniques for Yeast1 dataset

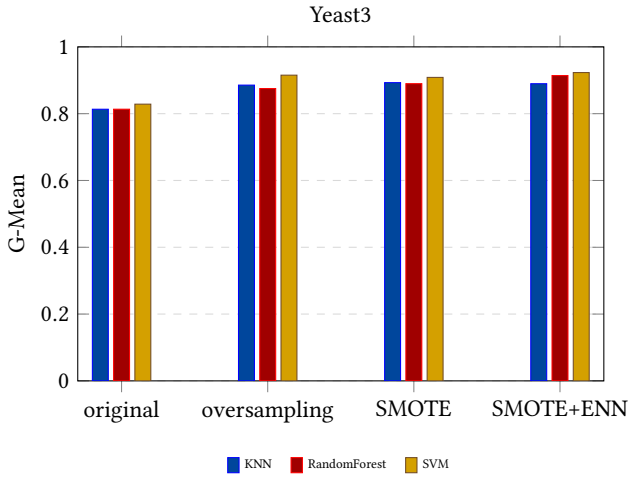

Figure 2: G-Mean between different techniques for Yeast3 dataset

4.2 Algorithm Level

4.3 Hybrid Method

5 Experimental Results Summary

This section provides a comprehensive summary of the experimental results across all evaluated methods and datasets.

6 Detailed Experimental Analysis

In this section, we provide a detailed analysis of the experimental results, categorized by the level of intervention: data-level, algorithm-level, and hybrid strategies.

Table 2: Summary table

Type	Method	Dataset	Acc.	Prec.	Rec.	F1	ROC-AUC	PR-AUC	G-Mean	BAcc.
Data Level	Baseline	yeast1	0.7581	0.6410	0.4025	0.4850	0.7881	0.5887	0.5976	0.6526
		yeast3	0.9477	0.8106	0.6848	0.7412	0.9570	0.7924	0.8184	0.8325
		yeast4	0.9661	0.3556	0.0970	0.1403	0.8561	0.3739	0.2219	0.5470
		yeast6	0.9798	0.4556	0.2952	0.3445	0.8999	0.5656	0.4296	0.6458
	Oversampling	yeast1	0.7280	0.5356	0.6596	0.5841	0.7835	0.5737	0.7021	0.7077
		yeast3	0.9292	0.6479	0.8488	0.7290	0.9509	0.7530	0.8920	0.8940
		yeast4	0.9261	0.3308	0.4897	0.3022	0.8732	0.3626	0.6272	0.7157
		yeast6	0.9537	0.3796	0.6857	0.4355	0.9080	0.4895	0.7952	0.8230
	SMOTE	yeast1	0.7201	0.5209	0.6760	0.5831	0.7834	0.5747	0.7029	0.7070
		yeast3	0.9328	0.6561	0.8551	0.7391	0.9571	0.7664	0.8971	0.8988
		yeast4	0.9115	0.2465	0.6152	0.3315	0.8801	0.3527	0.7406	0.7686
		yeast6	0.9452	0.3202	0.7333	0.4155	0.9240	0.4788	0.8248	0.8418
	SMOTE+ENN	yeast1	0.6734	0.4657	0.7948	0.5846	0.7803	0.5666	0.7011	0.7094
		yeast3	0.9216	0.6028	0.8938	0.7177	0.9533	0.7438	0.9089	0.9094
		yeast4	0.8821	0.1900	0.6921	0.2928	0.8881	0.3371	0.7769	0.7905
		yeast6	0.9306	0.2587	0.7810	0.3724	0.9324	0.4875	0.8468	0.8576
Algorithm Level	RandomForest	yeast1	0.7783	0.6650	0.4803	0.5553	0.8133	0.6435	0.6561	0.6899
		yeast3	0.9447	0.7870	0.6805	0.7275	0.9706	0.8316	0.8134	0.8289
		yeast4	0.9677	0.6000	0.1364	0.2062	0.9206	0.4073	0.3206	0.5668
		yeast6	0.9831	0.7667	0.4000	0.5054	0.8997	0.5629	0.6088	0.6986
	CS+RandomForest	yeast1	0.7891	0.6979	0.4825	0.5688	0.8114	0.6420	0.6632	0.6981
		yeast3	0.9461	0.7896	0.6992	0.7396	0.9726	0.8479	0.8254	0.8379
		yeast4	0.9650	0.3333	0.1164	0.1655	0.9269	0.4298	0.2572	0.5557
		yeast6	0.9838	0.8667	0.4286	0.5305	0.8965	0.5907	0.6350	0.7129
	AdaBoost(d=1)	yeast1	0.7601	0.6293	0.4219	0.5045	0.7878	0.5760	0.6152	0.6598
		yeast3	0.9508	0.7891	0.7725	0.7770	0.9721	0.7808	0.8664	0.8726
		yeast4	0.9623	0.3500	0.1945	0.2439	0.9217	0.3758	0.3814	0.5920
		yeast6	0.9831	0.7000	0.4857	0.5655	0.9380	0.5549	0.6836	0.7404
	CS+AdaBoost	yeast1	0.6658	0.4496	0.6922	0.5449	0.6736	0.4009	0.6729	0.6736
		yeast3	0.8956	0.5169	0.9203	0.6610	0.9470	0.7058	0.9061	0.9064
		yeast4	0.7224	0.2093	0.7436	0.3251	0.9190	0.2352	0.8154	0.8205
		yeast6	0.8956	0.1608	0.8000	0.2655	0.8972	0.2441	0.8374	0.8489
	XGBoost	yeast1	0.7648	0.6122	0.5151	0.5569	0.7847	0.6034	0.6662	0.6907
		yeast3	0.9434	0.7567	0.7176	0.7354	0.9612	0.7726	0.8338	0.8444
		yeast4	0.9629	0.4503	0.2764	0.3241	0.9165	0.3767	0.5024	0.6319
		yeast6	0.9771	0.5267	0.4857	0.4992	0.9246	0.5340	0.6875	0.7373
	CS+XGBoost	yeast1	0.7561	0.5765	0.5989	0.5861	0.7859	0.6012	0.6998	0.7094
		yeast3	0.9468	0.7482	0.7852	0.7649	0.9615	0.7895	0.8709	0.8760
		yeast4	0.9582	0.4230	0.3945	0.3932	0.9107	0.4053	0.6066	0.6865
		yeast6	0.9764	0.4984	0.5714	0.5294	0.9242	0.5515	0.7399	0.7788
Hybrid	AdaBoost(d=5)	yeast1	0.7540	0.5853	0.5196	0.5470	0.7821	0.5880	0.6612	0.6845
		yeast3	0.9421	0.7577	0.6928	0.7231	0.9591	0.7695	0.8199	0.8328
		yeast4	0.9650	0.5500	0.1964	0.2727	0.9312	0.4115	0.3923	0.5943
		yeast6	0.9778	0.5500	0.3714	0.4392	0.8979	0.4859	0.6043	0.6819
	RHSBoost	yeast1	0.7507	0.5613	0.6456	0.5993	0.7910	0.5896	0.7148	0.7195
		yeast3	0.9191	0.5901	0.9015	0.7115	0.9546	0.6852	0.9111	0.9114
		yeast4	0.8396	0.1545	0.8036	0.2584	0.8824	0.1807	0.8210	0.8223
		yeast6	0.8679	0.1359	0.8571	0.2340	0.8675	0.1328	0.8580	0.8627
	RUSBoost	yeast1	0.7156	0.5171	0.6085	0.5498	0.7558	0.5147	0.6728	0.6839
		yeast3	0.9003	0.5325	0.9203	0.6731	0.9348	0.5969	0.9090	0.9090
		yeast4	0.8255	0.1469	0.8418	0.2499	0.8847	0.1846	0.8321	0.8333
		yeast6	0.7864	0.0856	0.8286	0.1548	0.8265	0.0983	0.8009	0.8070
	SMOTEBoost	yeast1	0.7473	0.5560	0.6294	0.5903	0.7718	0.5543	0.7074	0.7123
		yeast3	0.9468	0.7530	0.7725	0.7611	0.9596	0.7989	0.8641	0.8704
		yeast4	0.9535	0.3799	0.3946	0.3730	0.9147	0.4122	0.6148	0.6840
		yeast6	0.9778	0.5101	0.6286	0.5539	0.9147	0.5500	0.7704	0.8074
	SMOTECSL	yeast1	0.5492	0.3855	0.9416	0.5470	0.7931	0.5956	0.6053	0.6656
		yeast3	0.8538	0.4260	0.9510	0.5883	0.9688	0.8093	0.8947	0.8964
		yeast4	0.6874	0.0877	0.8618	0.1591	0.8535	0.1488	0.7648	0.7715
		yeast6	0.7540	0.0865	0.9429	0.1573	0.9100	0.3574	0.8363	0.8462
	SUBoost	yeast1	0.7224	0.5297	0.5408	0.5277	0.7571	0.5586	0.6512	0.6685
		yeast3	0.9394	0.7743	0.6267	0.6877	0.9548	0.7286	0.7778	0.8024
		yeast4	0.9589	0.4417	0.2746	0.3146	0.8879	0.3961	0.4581	0.6289
		yeast6	0.9771	0.5343	0.4286	0.4462	0.8706	0.4566	0.6227	0.7095

Table 3: Classification outcomes (G-Mean) for kNN algorithm

Method	Yeast1	Yeast3	Yeast4	Yeast6
Original	0.6120	0.8133	0.3451	0.6801
Oversampling	0.6791	0.8855	0.7068	0.8233
SMOTE	0.6855	0.8929	0.7571	0.8434
SMOTE+ENN	0.7011	0.8897	0.8071	0.8624

Table 4: Classification outcomes (G-Mean) for random forest algorithm

Method	Yeast1	Yeast3	Yeast4	Yeast6
Original	0.6561	0.8134	0.3206	0.6088
Oversampling	0.7007	0.8752	0.3781	0.6752
SMOTE	0.7059	0.8898	0.6667	0.7705
SMOTE+ENN	0.7130	0.9141	0.7208	0.8056

Table 5: Classification outcomes (G-Mean) for SVM algorithm

Method	Yeast1	Yeast3	Yeast4	Yeast6
Original	0.5246	0.8285	0.0000	0.0000
Oversampling	0.7266	0.9153	0.7965	0.8870
SMOTE	0.7173	0.9086	0.7980	0.8606
SMOTE+ENN	0.6891	0.9231	0.8029	0.8724

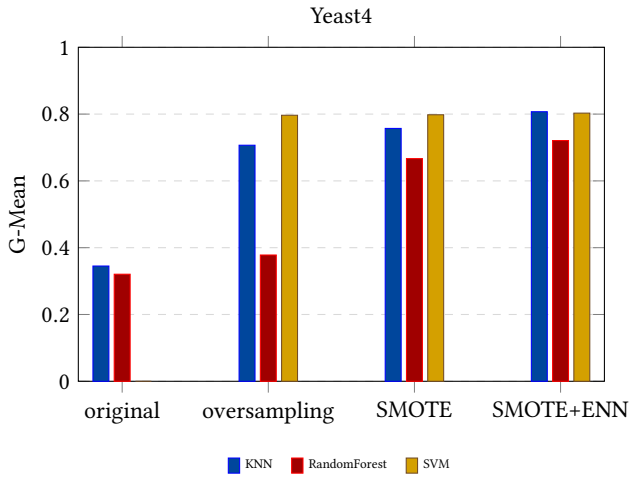

Figure 3: G-Mean between different techniques for Yeast4 dataset

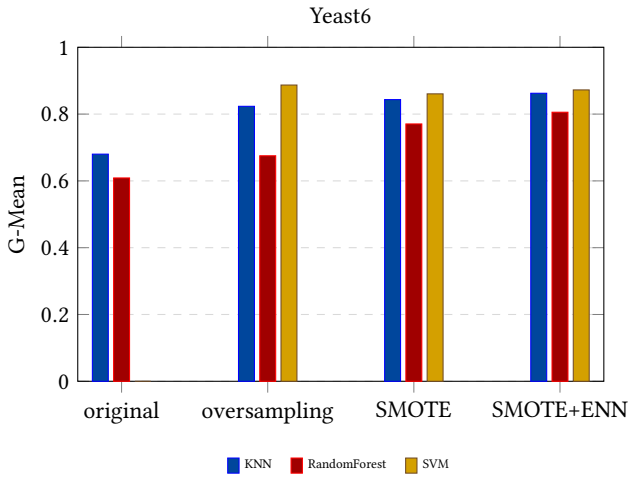

Figure 4: G-Mean between different techniques for Yeast6 dataset

6.1 Data Level Methods

Data-level methods focus on reslicing the training set to achieve a more balanced distribution. This section presenting the G-Mean performance for individual classifiers and visual analysis of the results.

6.2 Algorithm Level Methods

Algorithm-level methods, such as Cost-Sensitive learning combined with ensemble classifiers (Random Forest, XGBoost, AdaBoost), show higher robustness across varying imbalance ratios by directly internalizing the cost of misclassifying minority instances.

6.2.1 Visual Analysis of Algorithm-Level Methods. In this subcategory, we examine the performance shifts in ensemble methods when cost-sensitive mechanisms are applied.

Table 6: Summary of Experimental Results (Data-Level Methods)

Method	Dataset	Model	Acc.	Prec.	Rec.	F1	AUC	PR-AUC	G-Mean	BAcc
Original	yeast1	KNN	0.7392	0.5623	0.4360	0.4904	0.7550	0.5083	0.6120	0.6492
		RF	0.7783	0.6650	0.4800	0.5553	0.8133	0.6435	0.6561	0.6899
		SVM	0.7568	0.6956	0.2910	0.4092	0.7961	0.6142	0.5246	0.6187
	yeast3	KNN	0.9468	0.8119	0.6750	0.7362	0.9249	0.7188	0.8133	0.8277
		RF	0.9447	0.7870	0.6800	0.7275	0.9706	0.8316	0.8134	0.8289
		SVM	0.9515	0.8330	0.6990	0.7598	0.9755	0.8268	0.8285	0.8408
	yeast4	KNN	0.9650	0.4667	0.1550	0.2147	0.7944	0.2928	0.3451	0.5741
		RF	0.9677	0.6000	0.1360	0.2062	0.9206	0.4073	0.3206	0.5668
		SVM	0.9656	0.0000	0.0000	0.0000	0.8532	0.4215	0.0000	0.5000
	yeast6	KNN	0.9798	0.6000	0.4860	0.5281	0.8997	0.5167	0.6801	0.7387
		RF	0.9831	0.7667	0.4000	0.5054	0.8997	0.5629	0.6088	0.6986
		SVM	0.9764	0.0000	0.0000	0.0000	0.9003	0.6173	0.0000	0.5000
Oversampling	yeast1	KNN	0.6894	0.4727	0.6573	0.5498	0.7382	0.4774	0.6791	0.6799
		RF	0.7763	0.6244	0.5734	0.5964	0.8120	0.6383	0.7007	0.7161
		SVM	0.7183	0.5098	0.7481	0.6061	0.8003	0.6054	0.7266	0.7271
	yeast3	KNN	0.9131	0.5730	0.8525	0.6838	0.9151	0.6105	0.8855	0.8865
		RF	0.9488	0.7556	0.7917	0.7729	0.9666	0.8332	0.8752	0.8799
		SVM	0.9259	0.6151	0.9023	0.7303	0.9709	0.8153	0.9153	0.9156
	yeast4	KNN	0.9360	0.2700	0.5491	0.3585	0.7934	0.2407	0.7068	0.7494
		RF	0.9663	0.5444	0.1964	0.2624	0.9276	0.4124	0.3781	0.5950
		SVM	0.8760	0.1779	0.7236	0.2856	0.8987	0.4346	0.7965	0.8025
	yeast6	KNN	0.9589	0.3262	0.7143	0.4477	0.8954	0.3599	0.8233	0.8395
		RF	0.9805	0.5993	0.4857	0.5177	0.8890	0.5812	0.6752	0.7391
		SVM	0.9218	0.2134	0.8571	0.3411	0.9397	0.5274	0.8870	0.8903
SMOTE	yeast1	KNN	0.6934	0.4783	0.6691	0.5575	0.7447	0.4872	0.6855	0.6862
		RF	0.7642	0.5911	0.6013	0.5952	0.8043	0.6297	0.7059	0.7158
		SVM	0.7028	0.4932	0.7575	0.5965	0.8011	0.6070	0.7173	0.7190
	yeast3	KNN	0.9164	0.5817	0.8648	0.6944	0.9330	0.6695	0.8929	0.8938
		RF	0.9481	0.7363	0.8223	0.7766	0.9666	0.8186	0.8898	0.8930
		SVM	0.9340	0.6502	0.8780	0.7463	0.9715	0.8111	0.9086	0.9095
	yeast4	KNN	0.9036	0.2039	0.6473	0.3077	0.8303	0.2104	0.7571	0.7800
		RF	0.9522	0.3516	0.4745	0.3938	0.9107	0.3857	0.6667	0.7219
		SVM	0.8787	0.1839	0.7236	0.2929	0.8993	0.4619	0.7980	0.8039
	yeast6	KNN	0.9265	0.2144	0.7714	0.3344	0.9093	0.3032	0.8434	0.8509
		RF	0.9778	0.5167	0.6286	0.5573	0.9183	0.5391	0.7705	0.8074
		SVM	0.9313	0.2293	0.8000	0.3549	0.9444	0.5942	0.8606	0.8672
SMOTE+ENN	yeast1	KNN	0.6671	0.4570	0.8042	0.5828	0.7481	0.4748	0.7011	0.7078
		RF	0.7109	0.5007	0.7224	0.5903	0.7973	0.6146	0.7130	0.7143
		SVM	0.6422	0.4392	0.8577	0.5807	0.7954	0.6105	0.6891	0.7061
	yeast3	KNN	0.9003	0.5288	0.8771	0.6590	0.9222	0.6136	0.8897	0.8901
		RF	0.9394	0.6726	0.8837	0.7628	0.9666	0.8096	0.9141	0.9150
		SVM	0.9252	0.6070	0.9206	0.7312	0.9712	0.8083	0.9231	0.9232
	yeast4	KNN	0.8760	0.1826	0.7455	0.2929	0.8424	0.1908	0.8071	0.8131
		RF	0.9205	0.2305	0.5691	0.3253	0.9268	0.3933	0.7208	0.7511
		SVM	0.8497	0.1570	0.7618	0.2600	0.8951	0.4273	0.8029	0.8073
	yeast6	KNN	0.9036	0.1788	0.8286	0.2925	0.9077	0.2530	0.8624	0.8670
		RF	0.9676	0.3899	0.6857	0.4941	0.9457	0.5976	0.8056	0.8301
		SVM	0.9205	0.2073	0.8286	0.3305	0.9439	0.6119	0.8724	0.8756

Figure 5 presents a comparison of Recall scores between standard Random Forest and Weighted (Cost-Sensitive) Random Forest across the Yeast datasets.

Furthermore, Table **??** and Figure 6 highlight the "death cross" effect in AdaBoost. After applying cost-sensitive weights, Recall improves significantly, but at a severe cost to Precision.

Finally, the radar chart in Figure 7 illustrates the balanced improvement across multiple metrics (Recall, Precision, F1-Score, and G-Mean) for XGBoost when cost-sensitive learning is integrated.

6.3 Hybrid Methods

Hybrid methods integrate data resampling with algorithmic enhancements. This subsection evaluates the efficacy of boosting-based hybrid approaches.

6.3.1 Visual Analysis of Hybrid Methods. The hybrid framework combines data-level resampling with algorithm-level boosting. Figure 8 and Figure 9 provide insight into these combined effects.

Figure 8 illustrates the F1-score performance of various hybrid boosting methods across four Yeast datasets.

6.3.2 Impact of SMOTE Variants in Hybrid Frameworks. Figure 9 shows the comparative performance of SMOTE-based hybrid strategies. SMOTEBoost demonstrates superior F1-scores compared to the pure data-level SMOTE and the cost-sensitive variant SMOTECSL, particularly as the imbalance ratio increases in Yeast-4 and Yeast-6.

6.3.3 ROC Curve Analysis for Hybrid Frameworks. Figure 10 presents the ROC curves of various hybrid boosting methods across the four Yeast datasets. The curves illustrate the trade-off between the true positive rate and the false positive rate, with the area under the

Table 7: The complete experiment result of hybrid methods.

Method	Dataset	F1-score	ROC-AUC	G-Mean
AdaBoost(d=5)	yeast1	0.5470	0.7821	0.6612
	yeast3	0.7230	0.9591	0.8199
	yeast4	0.2727	0.9312	0.3923
	yeast6	0.4392	0.8979	0.6043
RHSBoost	yeast1	0.5993	0.7910	0.7148
	yeast3	0.7115	0.9546	0.9111
	yeast4	0.2584	0.8824	0.8210
	yeast6	0.2340	0.8675	0.8580
RUSBoost	yeast1	0.5498	0.7558	0.6728
	yeast3	0.6731	0.9348	0.9090
	yeast4	0.2499	0.8847	0.8321
	yeast6	0.1548	0.8265	0.8009
SMOTE	yeast1	0.5831	0.7834	0.7029
	yeast3	0.7391	0.9571	0.8971
	yeast4	0.3315	0.8801	0.7406
	yeast6	0.4155	0.9240	0.8248
SMOTEBoost	yeast1	0.5903	0.7718	0.7074
	yeast3	0.7611	0.9596	0.8641
	yeast4	0.3730	0.9147	0.6148
	yeast6	0.5539	0.9147	0.7704
SMOTECSL	yeast1	0.5470	0.7931	0.6053
	yeast3	0.5883	0.9688	0.8946
	yeast4	0.1591	0.8535	0.7648
	yeast6	0.1573	0.9100	0.8363
SUBoost	yeast1	0.5277	0.7571	0.6512
	yeast3	0.6877	0.9548	0.7778
	yeast4	0.3146	0.8879	0.4581
	yeast6	0.4462	0.8706	0.6227

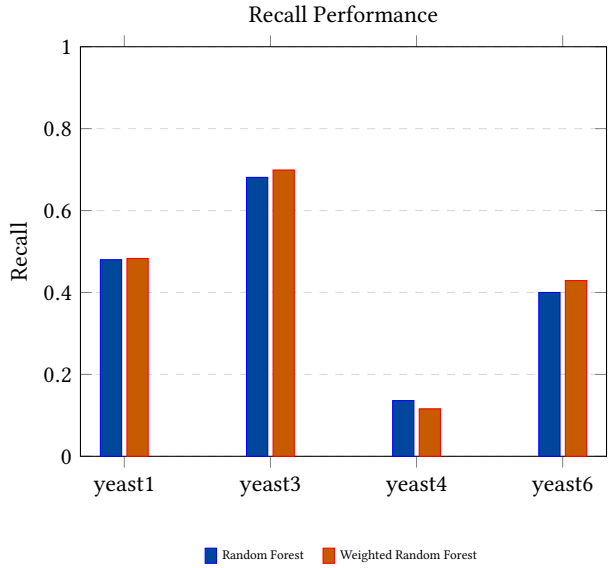

Figure 5: Comparison of Recall Performance between Baseline RF and Cost-Sensitive RF Across Datasets

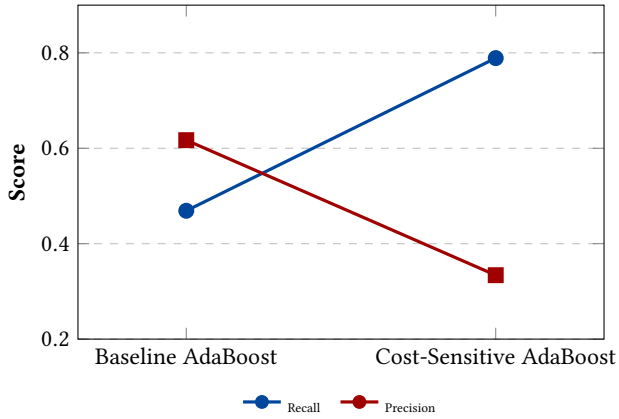

Figure 6: The extreme precision-recall trade-off in AdaBoost after applying the cost-sensitive mechanism.

curve (AUC) providing a metric for overall classification performance.

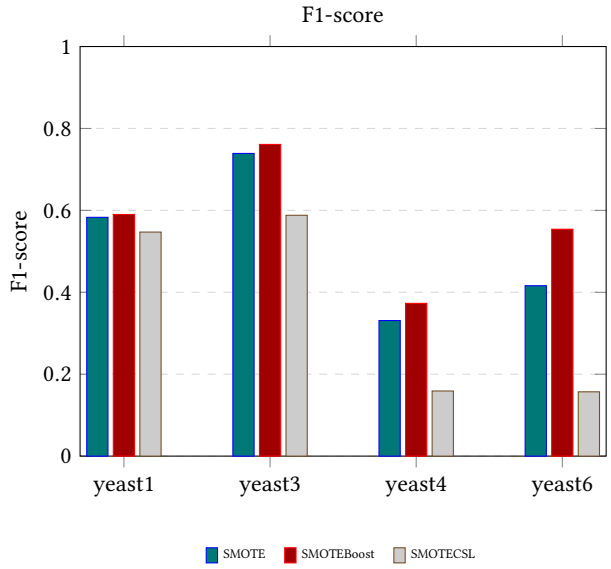

Figure 9: Experiment 2 F1-score Comparison Across Datasets.

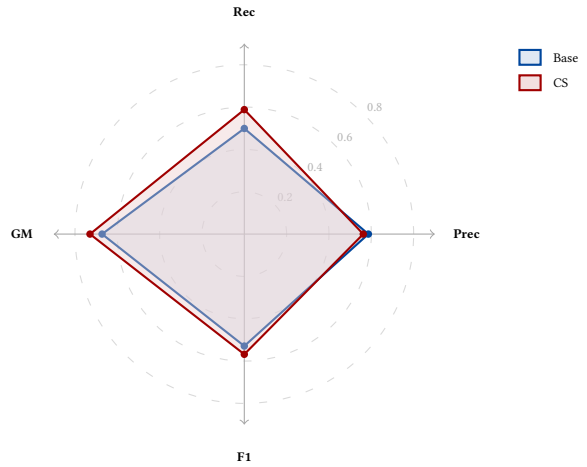

Figure 7: Radar Chart comparing the average performance of Baseline XGBoost and Cost-Sensitive XGBoost across all datasets.

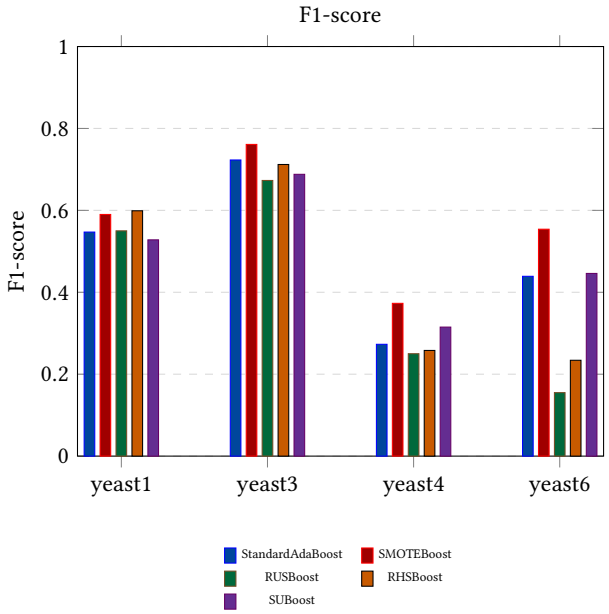

Figure 8: Experiment 1 F1-score Comparison Across Datasets.

6.4 Summary Statement

The experimental results demonstrate that while data-level methods provide a foundational improvement, algorithm-level and hybrid approaches offer superior stability and performance across extreme imbalance ratios.

References

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

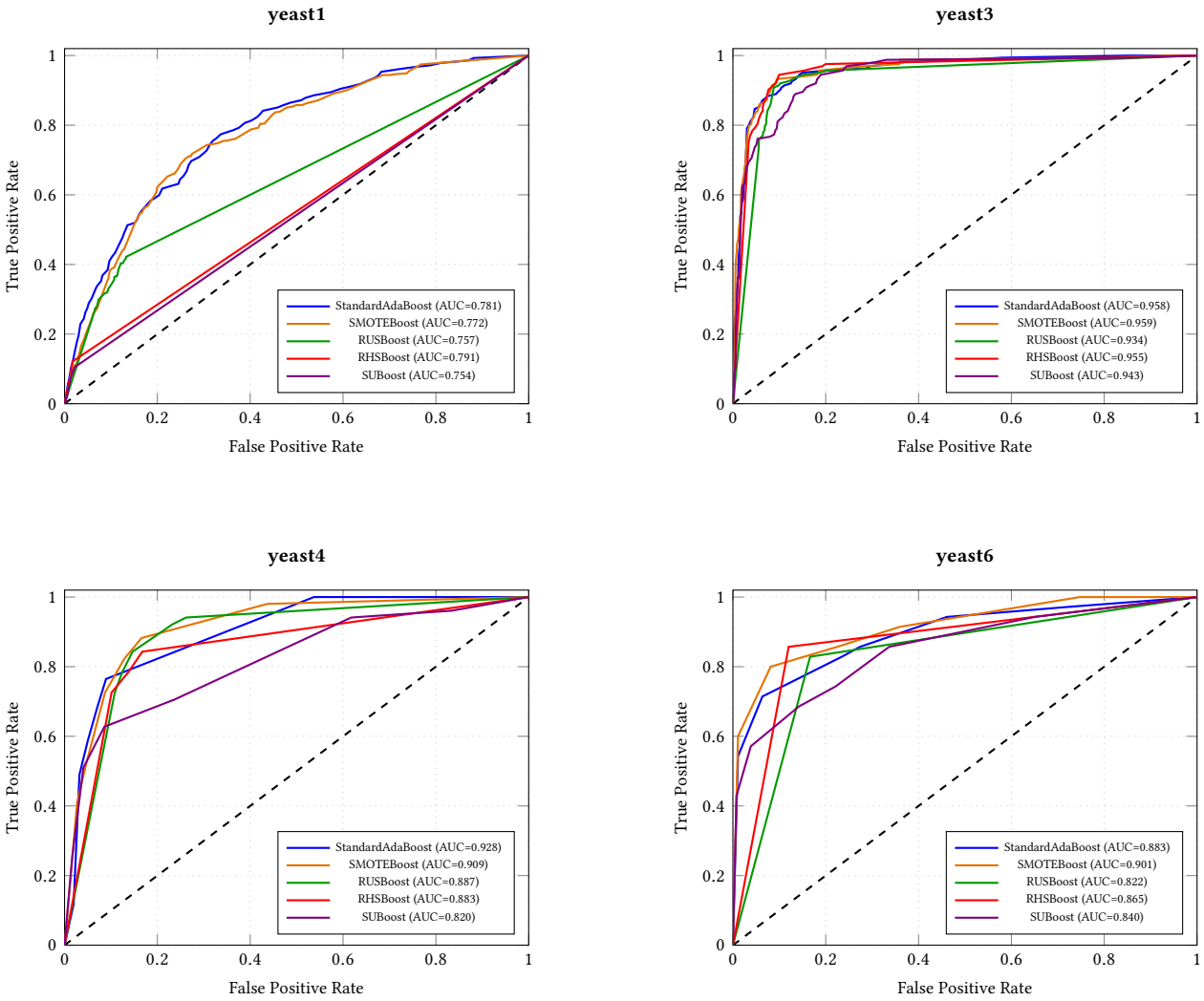

Figure 10: ROC Curves Comparison for Each Dataset.